Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure the Reinforcement Learning development environment using Python, Visual Studio Code, TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib, and verify the installation by executing a simple Reinforcement Learning program.

Procedure

1. Install Python and Visual Studio Code.
2. Open the project folder in VS Code.
3. Create and activate a virtual environment.
4. Upgrade the pip package manager.
5. Install the required libraries using the following command:

Bash

pip install numpy matplotlib gymnasium tensorflow keras

6. Create a Python file named test_rl_environment.py.
7. Write the verification program to import the required libraries and create a Gymnasium environment.
8. Execute the program using:

Bash

python test_rl_environment.py

9. Verify that all libraries are imported successfully and the CartPole-v1 environment is created without errors.

Python Code

Python

Run

import numpy as np

import matplotlib

import tensorflow as tf

from tensorflow import keras

import gymnasium as gym

```
print("=" * 60)
print("Reinforcement Learning Environment Verification")
print("=" * 60)
print("NumPy Version      :", np.__version__)
print("Matplotlib Version :", matplotlib.__version__)
print("TensorFlow Version :", tf.__version__)
print("Keras Version      :", keras.__version__)
env = gym.make("CartPole-v1")
observation, info = env.reset()
print("\nGymnasium Environment Created Successfully!")
print("Environment Name :", env.spec.id)
print("Initial State    :", observation)
env.close()
print("\nAll required libraries are installed successfully!")
```

Output

```
============================================================
Reinforcement Learning Environment Verification
============================================================
NumPy Version      : 2.5.1
Matplotlib Version : 3.11.0
TensorFlow Version : 2.21.0
Keras Version      : 3.15.0

Gymnasium Environment Created Successfully!
Environment Name : CartPole-v1
Initial State    : [ 0.00300641 -0.03977101  0.03911936  0.04487717]

All required libraries are installed successfully!
```

Result

The Reinforcement Learning development environment was successfully installed and configured. The required libraries were imported successfully, and the Gymnasium CartPole-v1 environment was created without any errors, confirming that the system is ready for Reinforcement Learning experiments.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Procedure

1. Import the Gymnasium library.
2. Create the FrozenLake-v1, CartPole-v1, and MountainCar-v0 environments.
3. Reset each environment to obtain the initial state.
4. Display the environment name and initial observation.
5. Perform a random action in each environment.
6. Observe the next state, reward, and episode status.
7. Close all environments after execution.

Python Code

Python

Run

```
import gymnasium as gym

# List of environments

environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]

for env_name in environments:

    print("=" * 60)

    print("Environment:", env_name)

    env = gym.make(env_name)

    observation, info = env.reset()

    print("Initial State:", observation)

    action = env.action_space.sample()

    print("Random Action Taken:", action)

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Next State:", next_state)
```

```
    print("Reward:", reward)
    print("Episode Ended:", terminated or truncated)
    env.close()
print("=" * 60)
print("Successfully interacted with all Gymnasium environments.")
```

Output

```
(rl_env) PS D:\DSA0335\RL> python experiment2.py
============================================================
Environment: FrozenLake-v1
Initial State: 0
Random Action Taken: 3
Next State: 0
Reward: 0
Episode Ended: False
============================================================
Environment: CartPole-v1
Initial State: [-0.04074467  0.03122096 -0.01882836 -0.03638627]
Random Action Taken: 1
Next State: [-0.04012025  0.22660777 -0.01955608 -0.33494976]
Reward: 1.0
Episode Ended: False
============================================================
Environment: MountainCar-v0
Initial State: [-0.5986676  0.       ]
Random Action Taken: 2
Next State: [-0.5971093   0.00155827]
Reward: -1.0
Episode Ended: False
============================================================
Successfully interacted with all Gymnasium environments.
```

Result

The FrozenLake-v1, CartPole-v1, and MountainCar-v0 environments were successfully created using the Gymnasium library. The initial state, random action, next state, reward, and episode status were observed, demonstrating the interaction between an agent and different Reinforcement Learning environments.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To develop a simple Markov Decision Process (MDP) model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Procedure

1. Define a set of states and actions.
2. Specify the transition function between states.
3. Assign rewards for each state transition.
4. Define a simple policy to select actions.
5. Simulate the movement of the agent through different states.
6. Display the current state, action taken, next state, and reward.
7. Stop the simulation when the terminal state is reached.

Python Code

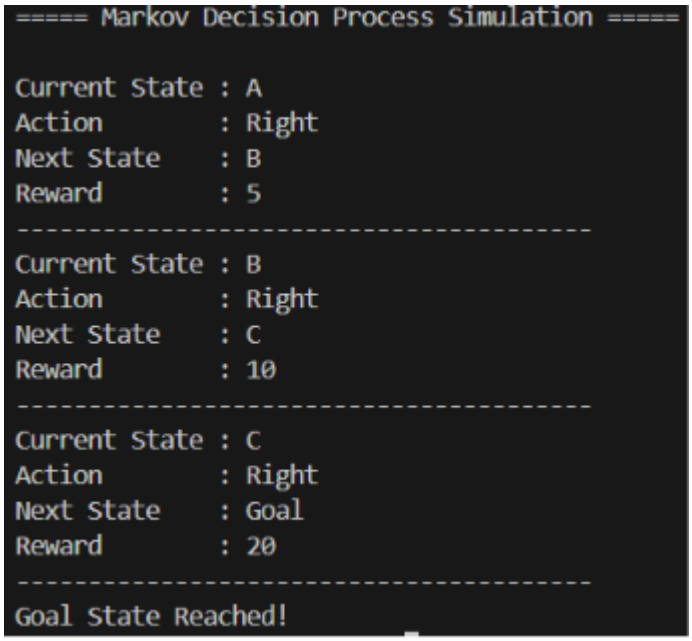
Python

Run

```
transitions = {
    "A": {"Right": ("B", 5)},
    "B": {"Right": ("C", 10)},
    "C": {"Right": ("Goal", 20)},
    "Goal": {}
}
current_state = "A"
print("Markov Decision Process Simulation\n")
while current_state != "Goal":
    action = "Right"
    next_state, reward = transitions[current_state][action]
    print(f"Current State : {current_state}")
    print(f"Action       : {action}")
```

```
    print(f"Next State    : {next_state}")
    print(f"Reward       : {reward}")
    print("-" * 40)
    current_state = next_state
print("Goal State Reached!")
```

Output



Result

The Markov Decision Process (MDP) was successfully simulated. The agent transitioned from the initial state to the goal state by following the defined policy, and the corresponding rewards were obtained at each state transition.

Experiment 6: Exploration vs. Exploitation Strategies

Aim

To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Procedure

1. Import the required Python library.
2. Define a set of reward values for different actions.
3. Implement the Random strategy to select an action randomly.

4. Implement the Greedy strategy to always choose the action with the highest reward.
5. Implement the ε-Greedy strategy using an epsilon (ε) value to balance exploration and exploitation.
6. Execute all three strategies and display the selected action and corresponding reward.
7. Compare the results obtained from each strategy.

Python Code

Python

Run

```
import random

rewards = [10, 25, 15, 40, 30]

print("Rewards:", rewards)

random_action = random.randint(0, len(rewards) - 1)

print("\n----- Random Strategy -----")

print("Selected Action:", random_action)

print("Reward:", rewards[random_action])

greedy_action = rewards.index(max(rewards))

print("\n----- Greedy Strategy -----")

print("Selected Action:", greedy_action)

print("Reward:", rewards[greedy_action])

epsilon = 0.2

if random.random() < epsilon:

    epsilon_action = random.randint(0, len(rewards) - 1)

else:

    epsilon_action = rewards.index(max(rewards))

print("\n----- Epsilon-Greedy Strategy -----")

print("Selected Action:", epsilon_action)

print("Reward:", rewards[epsilon_action])
```

Output

```
Rewards: [10, 25, 15, 40, 30]

----- Random Strategy -----
Selected Action: 4
Reward: 30

----- Greedy Strategy -----
Selected Action: 3
Reward: 40

----- Epsilon-Greedy Strategy -----
Selected Action: 3
Reward: 40
```

Result

The Random, Greedy, and ε-Greedy action-selection strategies were successfully implemented and executed. The Random strategy selected actions randomly, the Greedy strategy consistently chose the action with the highest reward, and the ε-Greedy strategy balanced exploration and exploitation by selecting either a random action or the best-known action based on the epsilon value.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim

To simulate a Reinforcement Learning agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Procedure

1. Import the Matplotlib library.
2. Define the rewards obtained in each episode.
3. Calculate the cumulative reward after every episode.
4. Plot the cumulative rewards using a line graph.
5. Label the graph with an appropriate title and axis labels.
6. Display the graph to visualize the learning performance of the agent.

Python Code

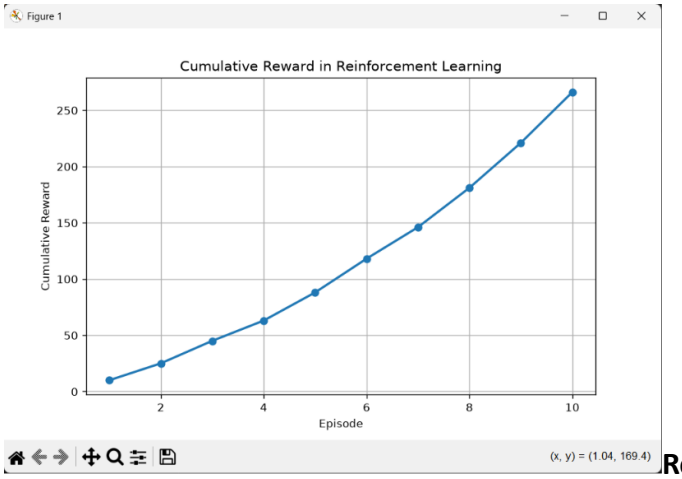
Python

Run

```
import matplotlib.pyplot as plt
```

```
episode_rewards = [10, 15, 20, 18, 25, 30, 28, 35, 40, 45]
cumulative_rewards = []
total_reward = 0
for reward in episode_rewards:
    total_reward += reward
    cumulative_rewards.append(total_reward)
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(cumulative_rewards) + 1),
         cumulative_rewards,
         marker='o',
         linewidth=2)
plt.title("Cumulative Reward in Reinforcement Learning")
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.grid(True)
plt.show()
```

Output



Result

The cumulative rewards obtained by the Reinforcement Learning agent were successfully visualized using a line graph. The graph illustrates the increase in cumulative rewards over multiple episodes, demonstrating the learning performance of the agent.